



Desarrollo de Aplicaciones II

Lilian Zapata

Índice

Inofirme técnico semana 6.....	3
Flujo funcional elegido.....	3
Mejoras Implementadas	4
Procesos Asincrónicos — Kotlin Coroutines.....	4
Debugging y Manejo de Errores.....	5
Diagnóstico y Corrección de Memory Leak — LeakCanary.....	7
Integración de Librería Externa — Retrofit 2.....	11
Justificación Técnica de Patrones Aplicados.....	13
Reflexión sobre Impacto en Calidad, Escalabilidad y UX.....	15

Informe Técnico — Semana 6

"Potenciando tu app con librerías externas"

Flujo Funcional Elegido

El flujo seleccionado es la carga y gestión de reservas de canchas deportivas, que comprende las siguientes operaciones:

- Carga de canchas desde una API REST externa al iniciar la app
- Visualización filtrada por tipo de deporte
- Reserva o liberación de una cancha mediante diálogo de confirmación
- Persistencia local del estado de cada reserva

Este flujo fue elegido por ser el núcleo funcional de la aplicación. Concentra las operaciones más críticas: conectividad de red, procesamiento de datos, persistencia local y actualización reactiva de la UI. Aplicar mejoras técnicas aquí impacta directamente en la experiencia del usuario y en la estabilidad del sistema.

Mejoras Implementadas

Procesos Asíncronos — Kotlin Coroutines

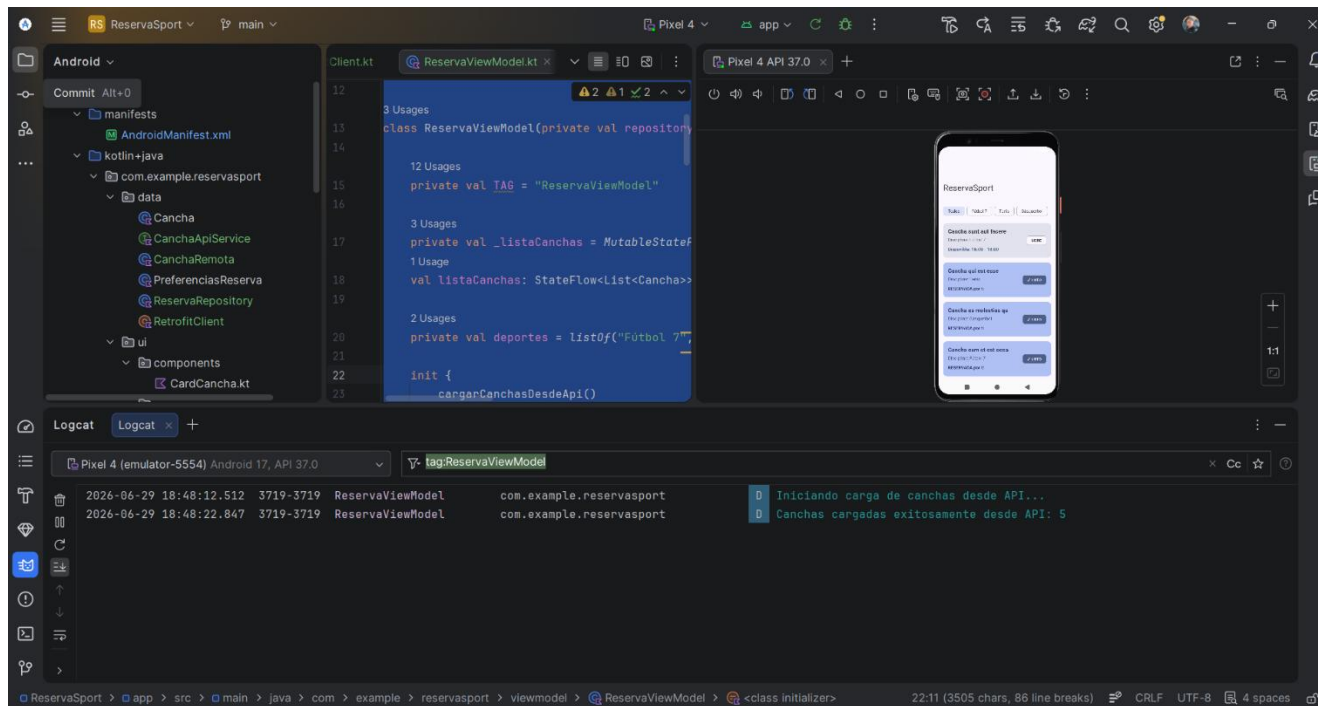
Para garantizar una experiencia fluida al usuario, se implementó Kotlin Coroutines en todas las operaciones que involucran red y almacenamiento.

La carga de canchas se ejecuta dentro de `viewModelScope.launch`. Elegiendo esta aproximación ya que el scope está vinculado al ciclo de vida del `ViewModel` — si el `ViewModel` es destruido, las operaciones pendientes se cancelan automáticamente, evitando que procesos huérfanos consuman recursos innecesariamente.

Las operaciones de lectura y escritura en `SharedPreferences` se delegan a `withContext(Dispatchers.IO)`, que las ejecuta en un pool de hilos separado del hilo principal. Esto es fundamental porque el hilo principal es el responsable de dibujar la interfaz — si se bloquea con operaciones de almacenamiento, la app se congela llevando a una mala experiencia para el usuario.

Adicionalmente, se implementó un mecanismo de fallback: si la API externa no responde, la app carga automáticamente un conjunto de datos locales predefinidos, garantizando que el usuario siempre tenga contenido disponible independientemente del estado de la conexión.

Imagen 1: Ejecución de procesos asíncronos con Kotlin Coroutines



Esta captura muestra el funcionamiento de las Coroutines dentro de la arquitectura MVVM, a la izquierda se pueden ver la estructura del proyecto, al centro se ve parte del código de ReservaViewModel. Y en el logcat se ve la ejecución exitosa de Coroutine con los mensajes descriptivos.

Debugging y Manejo de Errores

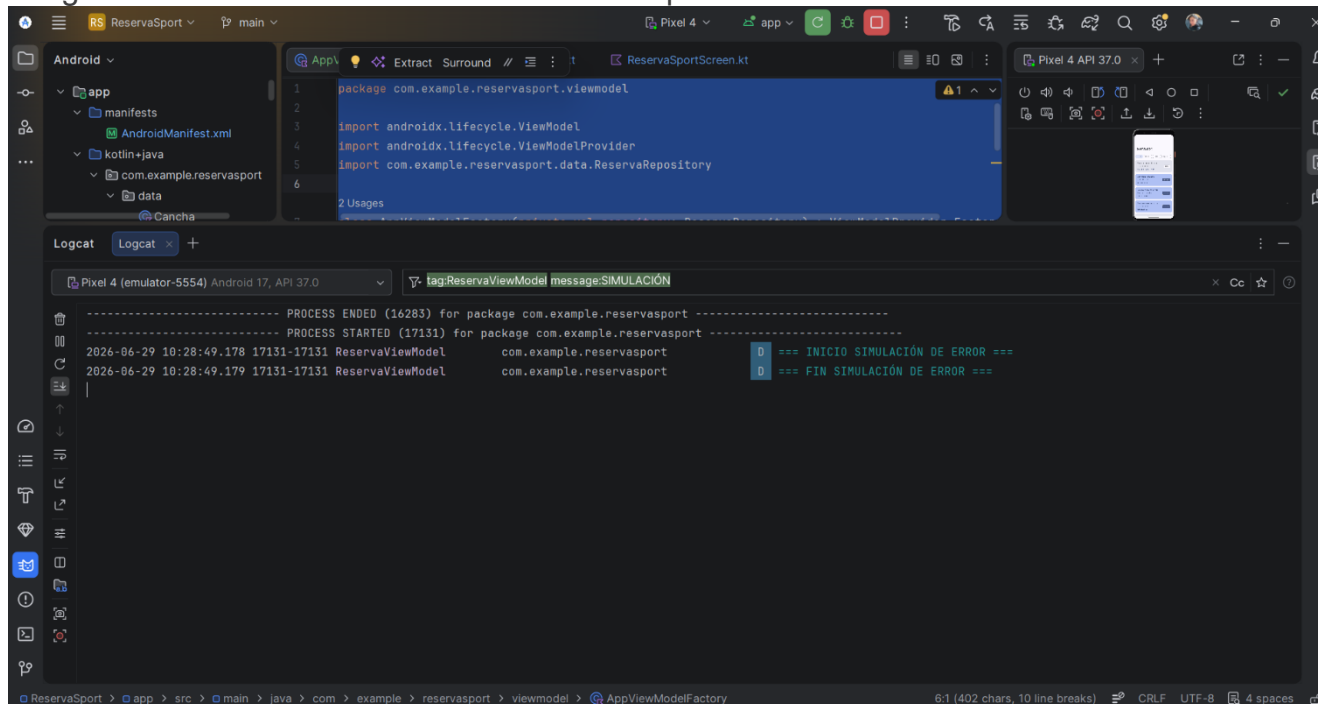
Se aplicaron bloques try-catch en las operaciones críticas de la aplicación, acompañados de registro en Logcat para mantener trazabilidad de los fallos.

En la llamada a Retrofit, se envolvió la petición a la API dentro de un try-catch. Si el servidor no responde o devuelve un error, el bloque catch registra el fallo con Log.e y activa automáticamente el fallback hacia los datos locales, evitando que el usuario se quede sin contenido.

En las operaciones de SharedPreferences se aplicó el mismo principio: si ocurre un error al leer o escribir el estado de una reserva, el catch registra el incidente y retorna un valor por defecto, evitando que la app se detenga por un fallo de persistencia.

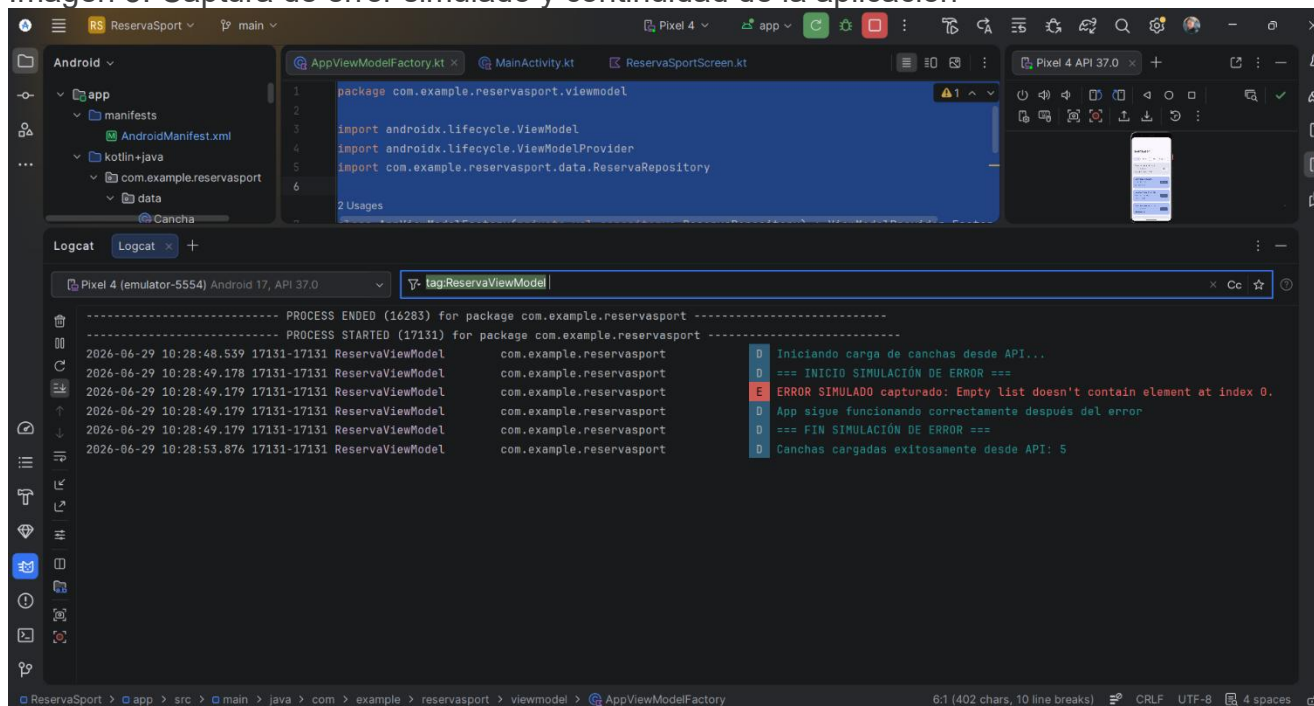
Para comprobar que el manejo de errores realmente funciona, se implementó la función `simularErrorDebugging()`, que provoca de forma controlada un `IndexOutOfBoundsException` al intentar acceder a un índice inexistente en una lista vacía. El bloque `catch` captura esta excepción y registra en Logcat que la aplicación continúa funcionando con normalidad después del error, demostrando que el manejo de excepciones está correctamente implementado y no compromete la estabilidad general de la app.

Imagen 2: Simulación controlada de error en operación crítica



Se puede observar el logcat filtrado por `tag:ReservaViewModel message:simulación`, logrando verse los mensajes descriptivos del error controlado que se lanzó. Permitiendo verificar que el manejo de errores mediante Try-Catch funciona bien antes de que ocurra una falla real.

Imagen 3: Captura de error simulado y continuidad de la aplicación



La captura posterior a ser filtrada muestra el flujo completo del error junto con el resto de la operación normal de la aplicación. Donde pese al error la estabilidad de la aplicación no se compromete de manera general.

Diagnóstico y Corrección de Memory Leak — LeakCanary

Herramienta utilizada: LeakCanary 2.14

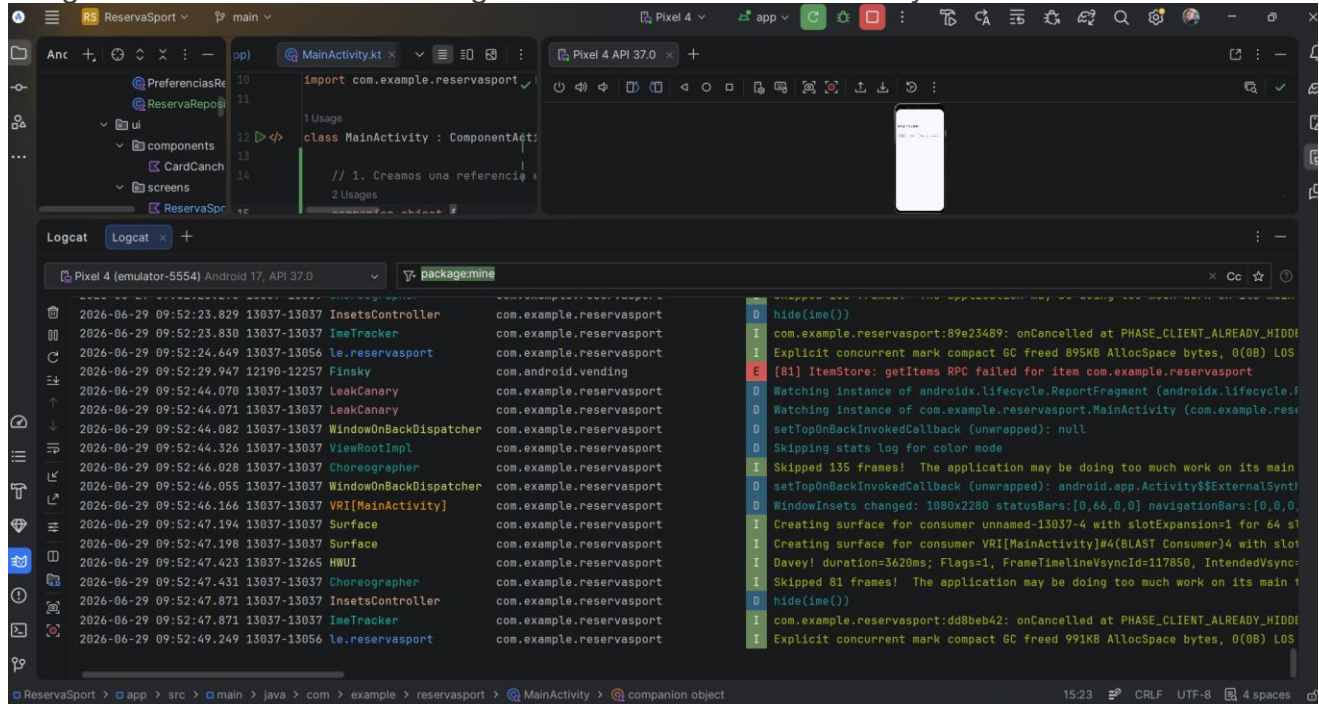
LeakCanary se integró en la dependencia debugImplementation, lo que significa que solo está activo en builds de debug y no afecta el rendimiento de producción. No requiere inicialización manual; se registra automáticamente al arrancar la aplicación.

Se creó deliberadamente una fuga de memoria en MainActivity asignando la instancia de la Activity a una variable estática dentro de un companion object. Una variable estática vive durante toda la ejecución de la app, por lo que al asignarle la referencia de la Activity (this), se generó una referencia fuerte que impidió al Garbage Collector liberarla cuando fue destruida, por ejemplo, al rotar la pantalla. Cada rotación generaba una nueva instancia de la Activity, pero la anterior nunca era liberada, acumulando memoria progresivamente.

Al rotar la pantalla durante las pruebas, LeakCanary detectó que la Activity había sido destruida pero no liberada por el recolector de basura, registrando en Logcat el monitoreo del objeto retenido.

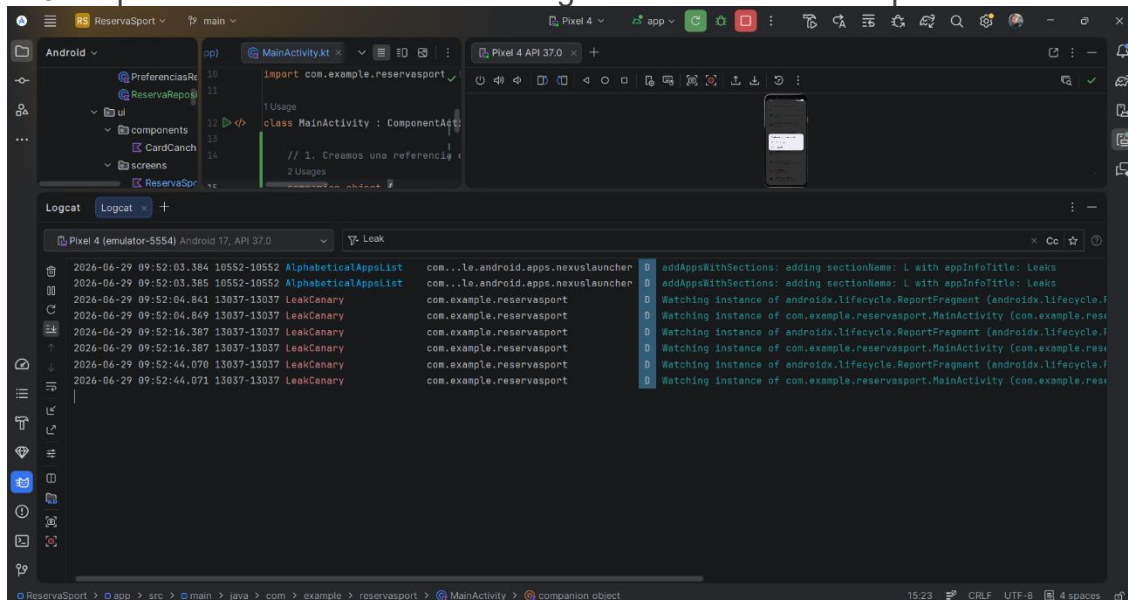
Posteriormente se corrigió el problema eliminando por completo el companion object y la asignación estática. Al retirar la referencia, el Garbage Collector pudo liberar correctamente cada instancia de la Activity al ser destruida, eliminando la fuga de memoria detectada.

Imagen 4: Detección inicial de fuga de memoria con LeakCanary



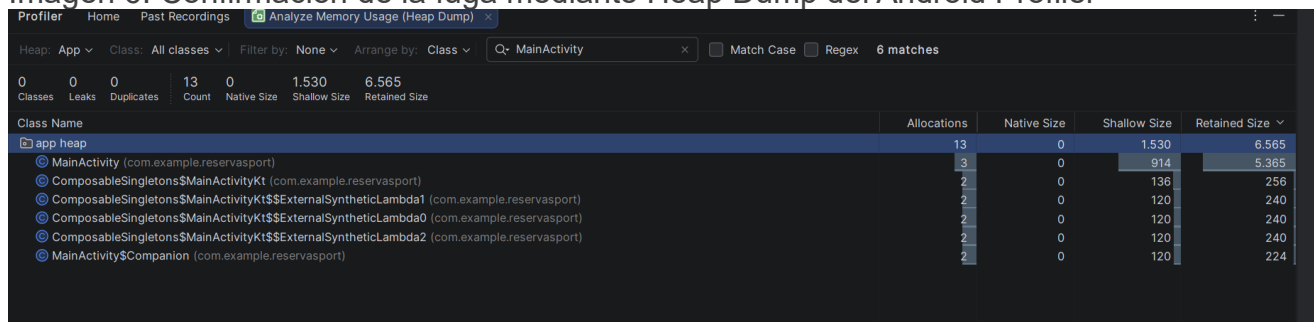
Esta imagen muestra el momento donde se detecta una fuga de memoria, lo que demuestra que LeakCanary identificó correctamente la instancia pese a que no fue liberada por Garbage Collector.

Imagen 5: Reproducción consistente de la fuga de memoria en múltiples ciclos



Se observa la fuga de memoria demostrando que no fue un evento aislado.

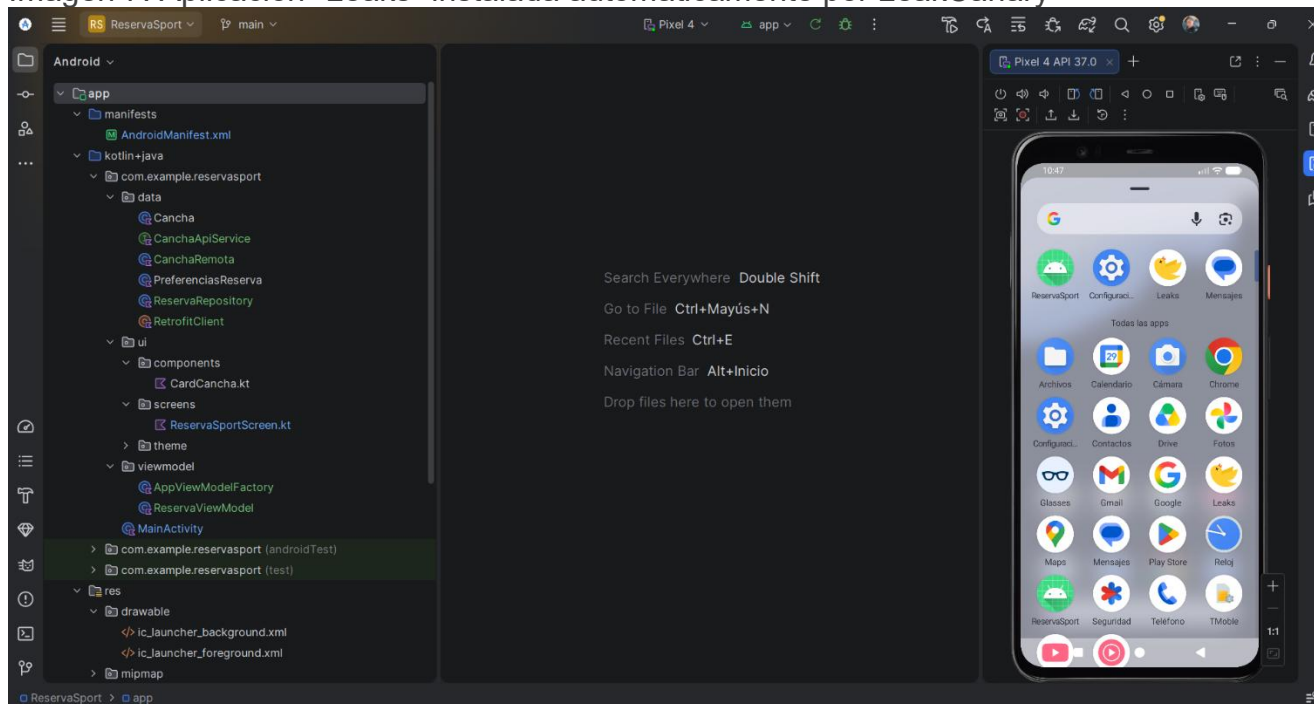
Imagen 6: Confirmación de la fuga mediante Heap Dump del Android Profiler



Class Name	Allocations	Native Size	Shallow Size	Retained Size
app heap	13	0	1,530	6,565
MainActivity (com.example.reservasport)	3	0	914	5,365
ComposableSingletons\$MainActivityKt (com.example.reservasport)	2	0	136	256
ComposableSingletons\$MainActivityKt\$\$ExternalSyntheticLambda1 (com.example.reservasport)	2	0	120	240
ComposableSingletons\$MainActivityKt\$\$ExternalSyntheticLambda0 (com.example.reservasport)	2	0	120	240
ComposableSingletons\$MainActivityKt\$\$ExternalSyntheticLambda2 (com.example.reservasport)	2	0	120	240
MainActivity\$Companion (com.example.reservasport)	2	0	120	224

Corresponde al análisis de memoria generado por Profiler, el cual complementa la evidencia de LeakCanary, donde se pueden observar tres instancias, en vez de ser liberadas tras su destrucción.

Imagen 7: Aplicación "Leaks" instalada automáticamente por LeakCanary

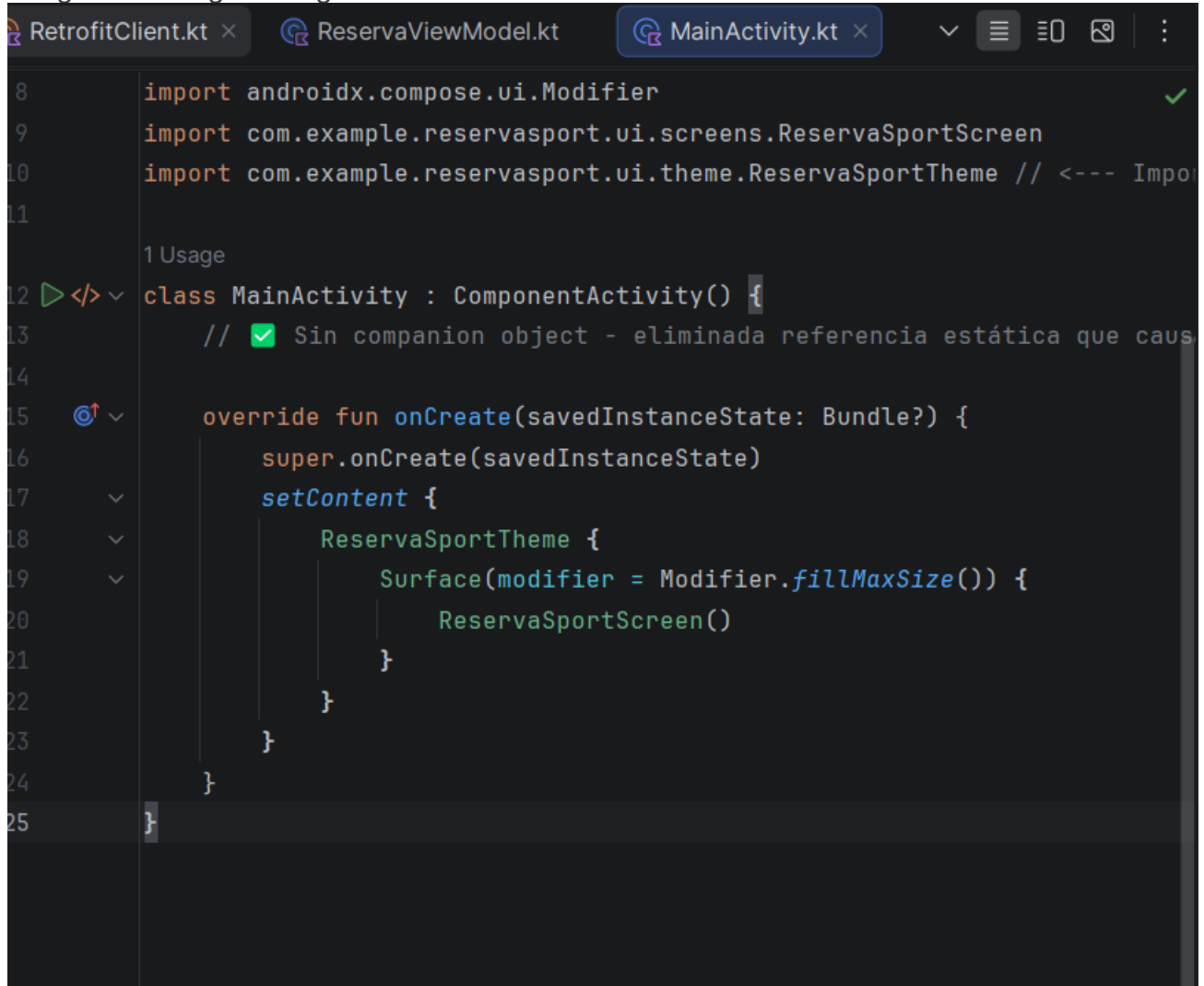


Se puede ver en el emulador, la existencia de la aplicación Leak (icono de pájaro amarillo)

Corrección aplicada:

Al eliminar la referencia estática, el Garbage Collector puede liberar correctamente cada instancia de la Activity al ser destruida, eliminando la fuga de memoria.

Imagen 8: Código corregido



```
8      import androidx.compose.ui.Modifier
9      import com.example.reservasport.ui.screens.ReservaSportScreen
10     import com.example.reservasport.ui.theme.ReservaSportTheme // <--- Import
11
12     1 Usage
13     class MainActivity : AppCompatActivity() {
14         // ✓ Sin companion object - eliminada referencia estática que caus
15
16         override fun onCreate(savedInstanceState: Bundle?) {
17             super.onCreate(savedInstanceState)
18             setContent {
19                 ReservaSportTheme {
20                     Surface(modifier = Modifier.fillMaxSize()) {
21                         ReservaSportScreen()
22                     }
23                 }
24             }
25         }
26     }
```

Esta imagen muestra como queda el MainActivity luego de la corrección al eliminar companion object que contenía la variable estática fugaDeMemoria.

Integración de Librería Externa — Retrofit 2

Librería integrada: Retrofit 2.11.0 + Gson Converter

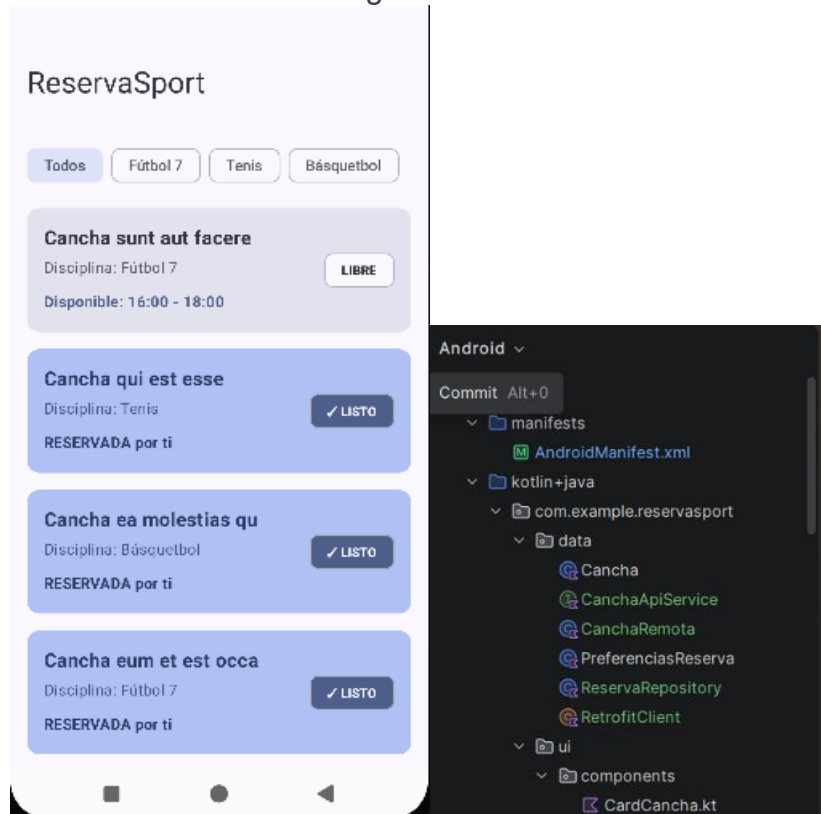
Justificación técnica:

Se seleccionó Retrofit como librería externa por ser el estándar de la industria para consumo de APIs REST en Android, desarrollado por Square. Entre sus ventajas técnicas destacan: la posibilidad de definir endpoints como interfaces Kotlin mediante anotaciones (@GET, @POST), separando así la definición del contrato HTTP de su implementación; su integración nativa con Kotlin Coroutines a través de funciones suspend, lo que elimina la necesidad de callbacks anidados; la deserialización automática de JSON a objetos Kotlin mediante Gson Converter; y la centralización de la configuración HTTP en un único objeto (RetrofitClient), aplicando el patrón Singleton.

Implementación:

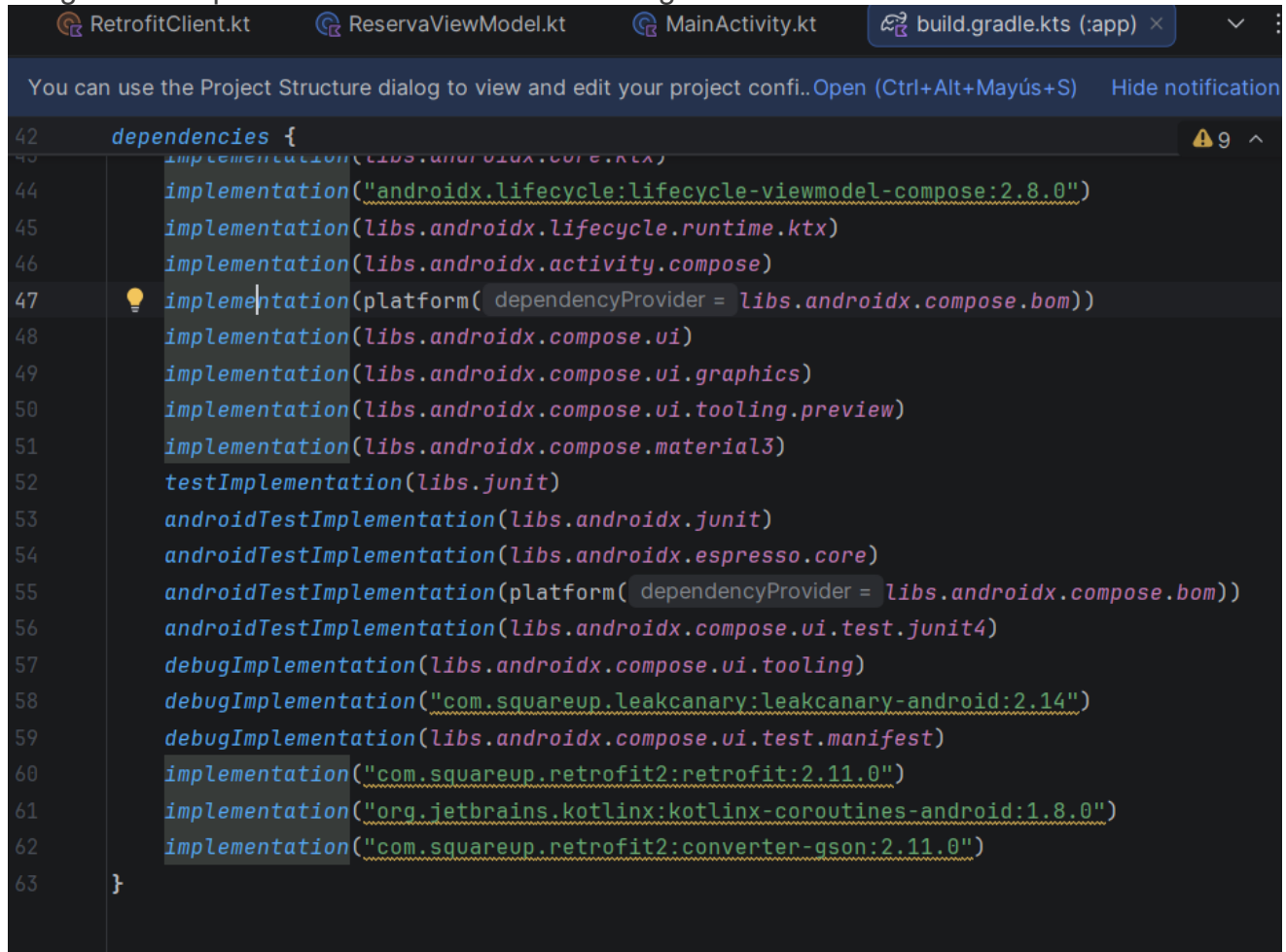
Se consumió la API pública JSONPlaceholder (<https://jsonplaceholder.typicode.com/posts>) como servidor de simulación, dado que la aplicación no cuenta con un backend propio en esta etapa de desarrollo. Los títulos de los posts retornados por la API se utilizaron como nombres de canchas, demostrando así la integración completa del flujo de red: solicitud, recepción, deserialización y presentación de datos en pantalla.

Imagen 9: Verificación visual de datos cargados desde Retrofit



Se puede observar que al estar en ejecución la aplicación, las canchas se ven cargadas exitosamente desde la API, confirmando que la integración de la librería externa funciona correctamente de extremo a extremo. Y a la derecha, se puede observar el código RetrofitClient.

Imagen 10: Dependencias declaradas en build.gradle.kts



```
42 dependencies {
43     implementation(libs.androidx.core.ktx)
44     implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
45     implementation(libs.androidx.lifecycle.runtime.ktx)
46     implementation(libs.androidx.activity.compose)
47     implementation(platform(dependencyProvider = libs.androidx.compose.bom))
48     implementation(libs.androidx.compose.ui)
49     implementation(libs.androidx.compose.ui.graphics)
50     implementation(libs.androidx.compose.ui.tooling.preview)
51     implementation(libs.androidx.compose.material3)
52     testImplementation(libs.junit)
53     androidTestImplementation(libs.androidx.junit)
54     androidTestImplementation(libs.androidx.espresso.core)
55     androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
56     androidTestImplementation(libs.androidx.compose.ui.test.junit4)
57     debugImplementation(libs.androidx.compose.ui.tooling)
58     debugImplementation("com.squareup.leakcanary:leakcanary-android:2.14")
59     debugImplementation(libs.androidx.compose.ui.test.manifest)
60     implementation("com.squareup.retrofit2:retrofit:2.11.0")
61     implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.8.0")
62     implementation("com.squareup.retrofit2:converter-gson:2.11.0")
63 }
```

Se puede observar el build.gradle.kts del modulo app, donde se evidencian las dependencias agregadas necesarias para la integración de la librería externa y el manejo de los proceso asíncronos.

Justificación Técnica de Patrones Aplicados

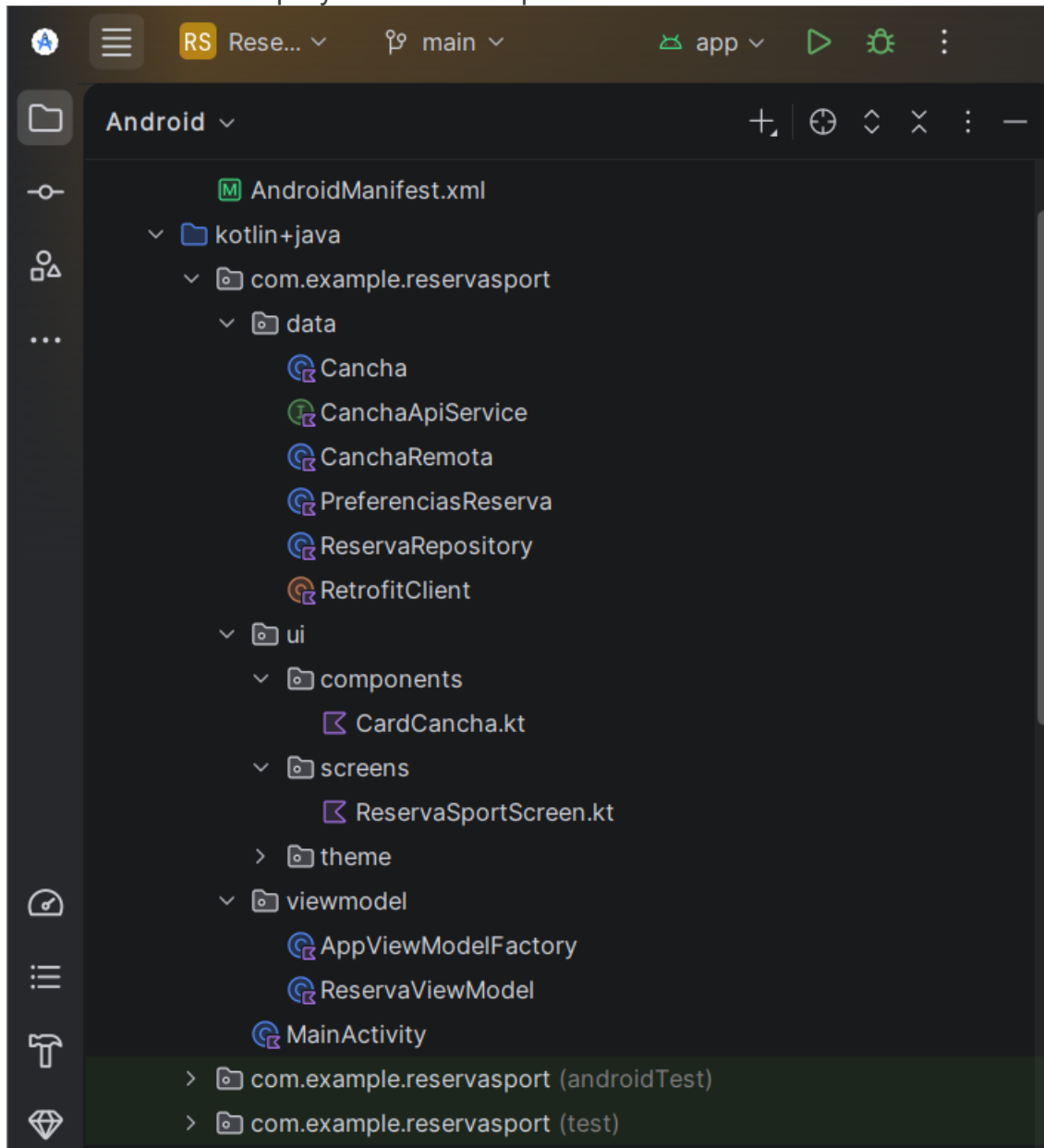
Se aplicó el patrón MVVM, separando la lógica de negocio (ViewModel) de la interfaz (Composables) y de los datos (Repository). El ViewModel expone el estado mediante StateFlow, un flujo observable y reactivo que permite que la UI se actualice automáticamente ante cualquier cambio de estado, sin necesidad de acoplamiento directo entre capas.

Se implementó además el patrón Repository, donde ReservaRepository abstrae las fuentes de datos detrás de una interfaz única. Gracias a esto, el ViewModel desconoce si los datos

proviene de la red o del almacenamiento local, lo que facilita reemplazar la fuente de datos en el futuro sin modificar la lógica de presentación.

Finalmente, el uso de Kotlin Coroutines con `viewModelScope` garantiza que todas las coroutines lanzadas se cancelen automáticamente cuando el `ViewModel` es destruido, previniendo fugas de memoria asociadas a operaciones de red o persistencia que quedarían pendientes sin un receptor activo.

Imagen 11: Estructura del proyecto con la arquitectura MVVM



Se puede observar cómo es la organización del proyecto bajo el patrón MVVM, con separación clara de responsabilidades según el folder. Lo cual facilita el mantenimiento, escalabilidad y separación entre lógica de negocio, interfaz y datos.

Reflexión sobre Impacto en Calidad, Escalabilidad y UX

Calidad: La combinación de try-catch con logging estructurado en Logcat permite diagnosticar fallos en producción con trazabilidad completa. LeakCanary actúa como red de seguridad durante el desarrollo, detectando fugas antes de que lleguen a los usuarios (aunque solo logre ver en logcat, ni me salió notificación en la misma aplicación para ver el informe).

Escalabilidad: La arquitectura MVVM con Repository Pattern permite reemplazar JSONPlaceholder por una API real de backend sin modificar el ViewModel ni la UI. Basta con actualizar RetrofitClient y CanchaApiService. Del mismo modo, SharedPreferences puede reemplazarse por Room Database sin impactar las capas superiores.

Experiencia de usuario: Al ejecutar todas las operaciones pesadas en hilos de background con Coroutines, la UI permanece fluida y responsiva en todo momento, eliminando el riesgo de ANR. El mecanismo de fallback a datos locales garantiza que el usuario siempre vea contenido, incluso si la conexión a internet falla (si fuera mejor mi notebook, se lograría mayor fluidez).



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.